

```
from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.orm import DeclarativeBase, Mapped,
mapped_column
from sqlalchemy import String, Numeric
from decimal import Decimal
from flask import Flask
from flask_cors import CORS
```

```
app = Flask(__name__)
CORS(app)
```

Afficher plus ▾

Ton code est globalement correct et fonctionnel, mais il contient quelques erreurs, des imports inutiles et des améliorations possibles. Voici une revue détaillée.

1. Imports

Ton code

```
from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy import String, Numeric
from decimal import Decimal
```

</> Python



▶ Exécuter

Problèmes

- `Flask` est importé deux fois.
- `Numeric` n'est jamais utilisé.
- `Decimal` n'est jamais utilisé.

Tu peux simplement écrire :

```
from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
from flask_cors import CORS
```

</> Python



Exécuter

2. Création de l'application

Tu fais deux fois :

```
app = Flask(__name__)
CORS(app)
```

</> Python



Exécuter

La première création est immédiatement écrasée par la seconde.

Il faut garder uniquement :

</> Python



Exécuter

3. Configuration SQLAlchemy

Correct :

</> Python



Exécuter

Tu peux également ajouter :

</> Python



Exécuter

pour éviter un avertissement.

4. Modèle Trip

Très bien.

En revanche le prix devrait idéalement être un nombre décimal.

Au lieu de

</> Python



▶ Exécuter

on préfère

```
from sqlalchemy import Numeric
```

</> Python



▶ Exécuter

ou, si tu veux rester simple avec SQLite :

</> Python



▶ Exécuter

5. Fonction to_dict()

Correct.

Si tu utilises `Decimal`, il faudra convertir :

</> Python



▶ Exécuter

Sinon `jsonify` ne saura pas sérialiser un `Decimal`.

6. Création des tables

Correct.

Python



Exécuter

7. Route "/"

Correct.

```
@app.route("/")
```

Python



Exécuter

8. GET /trips

Très bien.

Python



Exécuter

Puis

Python



Exécuter

Excellent.

9. GET /trips/<id>

Correct.

Python



Exécuter

Puis

 Python Exécuter

Très bien.

10. Les triples guillemets

Tu as :

```
.....
```

 Python Exécuter

Ils ne servent à rien.

Supprime-les.

11. POST /trips

Validation JSON

Correct :

 Python Exécuter

Validation du titre

Correct.

 Python Exécuter

Puis

 Python Exécuter

Très bien.

Validation des champs

Tu utilises :

Python



▶ Exécuter

C'est une bonne idée.

Petit problème

Tu vérifies :

Python



▶ Exécuter

Puis plus bas :

Python



▶ Exécuter

Mais juste avant tu imposes déjà que le champ existe.

Donc :

Python



▶ Exécuter

ne sert à rien.

Tu peux écrire directement :

Python



▶ Exécuter

ou alors rendre ce champ facultatif en l'enlevant de `mandatory_fields`.

Par exemple :

```
mandatory_fields = {  
    "title": str,  
    "destination": str,  
    "country": str,  
    "description": str,  
    "price": (int, float),  
    "duration": int,  
    "travel_type": str,  
}
```

</> Python



▶ Exécuter

et

</> Python



▶ Exécuter

12. Validation du prix

Tu acceptes :

</> Python



▶ Exécuter

Très bien.

Mais dans SQLAlchemy tu stockes un float.

Si quelqu'un envoie

</> JSON



la validation échoue, ce qui est normal.

13. Validation du code pays

Tu ne vérifies pas sa longueur.

Tu pourrais ajouter

```
country = body["country"]
```

Python



Exécuter

14. Validation de travel_type

Tu pourrais vérifier qu'il appartient à une liste.

Exemple

```
allowed = ["city", "nature", "beach", "adventure"]
```

Python



Exécuter

15. Validation de duration

Tu vérifies uniquement le type.

Tu pourrais ajouter

ChatGPT Mettre à niveau



Python



Exécuter

16. Validation du prix

Même chose.

Python



Exécuter

```
if body["price"] <= 0:  
    return jsonify({"error": "Prix invalide"}), 400
```

17. Retour HTTP

Très bien.

Python



Exécuter

```
return jsonify(trip.to_dict()), 201
```

Le code **201 Created** est exactement celui qu'il faut.

18. Structure générale

Je réorganiserais légèrement le fichier :

Imports



↓

Création app

↓

Configuration

↓

Base SQLAlchemy

↓

Modèles

↓

create_all()

↓

Routes GET

↓

Routes POST

↓

Routes PUT

↓

Routes DELETE

↓

`app.run()`

C'est la structure la plus courante.

19. Note globale

Je donnerais les notes suivantes :

Partie	Note
Structure	★★★★★ (5/5)
SQLAlchemy	★★★★★ (5/5)
Routes GET	★★★★★ (5/5)
Route POST	★★★★☆ (4/5)
Validation	★★★★☆ (4/5)
Lisibilité	★★★★★ (5/5)

Note globale : 18,5/20.

Les principales corrections à apporter sont :

- supprimer les imports inutiles (`Numeric` , `Decimal` , doublon de `Flask`) ;
- ne créer `app` qu'une seule fois ;
- supprimer les triples guillemets inutiles ;
- ajouter `SQLALCHEMY_TRACK_MODIFICATIONS = False` ;
- rendre `is_available` soit réellement obligatoire, soit réellement optionnel (actuellement la logique est redondante) ;
- renforcer les validations (code pays sur 2 caractères, prix > 0, durée > 0, valeurs autorisées pour `travel_type`). Ces améliorations rendront ton API plus robuste et

plus proche des bonnes pratiques professionnelles.

